

Platform Devices und Device Tree

4.05

Vier Bausteine trennen Hardware und Treibercode

01 Hardwarebeschreibung

DeviceTreeSource (DTS) Knoten beschreiben vorhandene Hardware und ihre Ressourcen.

02 Instanziierung

Der OpenFirmware Core verarbeitet den DTS-Knoten erzeugt Geräte Objekte.

03 Matching

`compatible` verbindet Hardware Instanz und Treiber.

04 Treiberlebenszyklus

`probe` initialisiert; `remove` und Shutdown stoppen die Nutzung.

Device Tree beschreibt Hardware · der Treiber implementiert Verhalten

Fünf Ebenen führen vom Board zum Treiber

Stufe	Form	Verwendung
DTS	Textuelle Hardwarebeschreibung	Quelle für Board und SoC
DTB	Kompilierter Binärbaum	Übergabe durch Firmware
OF-Core	Live Device Tree	Knoten lesen und populieren
platform_device	Geräteobjekt mit Ressourcen	Teil des Linux Device Model
platform_driver	Match-Tabelle und Callbacks	Hardware initialisieren

Hardwaredaten bleiben im Device Tree; Verhalten bleibt im Treiber.

Vom DTS-Knoten bis zum gebundenen Platform-Treiber



Beschreibung Knoten und Properties modellieren die Hardware

Übergabe Bootloader oder Firmware liefert den DTB

Population Der OF-Core erzeugt Platform Devices

Bindung

Der Driver Core ruft beim Match probe() auf.

Ein Device-Tree-Knoten beschreibt eine Hardwareinstanz

Element	Bedeutung	Beispiel
Knotenname	Typ und Unit-Adresse	demo@40000000
compatible	stabile Hardwarekennung	acme,demo-v1
reg	Adress- und Größenzellen	<0x40000000 0x1000>
interrupts	Interrupt-Spezifikation	<42>
status	Knoten aktivieren	okay / disabled

Properties beschreiben Hardwareeigenschaften – nicht die Implementierung des Linux-Treibers.

Ein Device-Tree-Knoten beschreibt nur Hardware

```
demo@40000000 {  
    compatible = "acme,demo-v1";  
    reg = <0x40000000 0x1000>;  
    interrupts = <42>;  
    clocks = <&clk 3>;  
    status = "okay";  
};
```

- | | | |
|-----------|-------------------|------------------------------------|
| 01 | compatible | stabile Kennung für das Matching |
| 02 | reg | MMIO-Adresse und Größe |
| 03 | interrupts | IRQ über den Interrupt-Controller |
| 04 | clocks | Referenz auf Taktgeber und Kennung |
| 05 | status | okay erzeugt ein verfügbares Gerät |

Der Knoten beschreibt Verdrahtung und Ressourcen; Laufzeitverhalten gehört in den Treiber.

DTS, DTB und Live Tree sind drei Darstellungen

DTS

- menschenlesbare Quelle
- Includes und Labels
- Board-Varianten
- Versionskontrolle



DTB

- mit dtc kompiliert
- binäres Baumformat
- Bootloader übergibt
- Kernel liest früh



Live Tree

- struct device_node
- OF-APIs lesen Properties
- /sys/firmware/devicetree
- Basis für Population

DTS ist die Quelle, DTB das Transportformat und der Live Tree die Kernelrepräsentation.

Fünf Regeln halten Device Trees portabel

Hardware statt Policy

Beschreiben, was vorhanden und verdrahtet ist.

stabiles compatible

Herstellerpräfix und konkrete Gerätefamilie nutzen.

Standard-Properties

reg, interrupts, clocks und resets bevorzugen.

Zellen korrekt kodieren

#address-cells und #size-cells des Parents beachten.

Schema prüfen

Binding und DTS automatisiert validieren.

Device-Tree-Bindings bilden eine ABI und müssen über Kernelversionen hinweg verständlich bleiben.

Bindings machen die Hardwarebeschreibung prüfbar

```
$ id: http://devicetree.org/schemas/misc/acme,demo.yaml#  
$ schema: http://devicetree.org/meta-schemas/core.yaml#  
  
title: ACME Demo Controller  
  
properties:  
  compatible:  
    const: acme,demo-v1  
  reg:  
    maxItems: 1  
  
required: [compatible, reg]  
additionalProperties: false
```

- | | | |
|-----------|----------------------|--|
| 01 | Identität | \$id und title ordnen das Schema zu |
| 02 | Eigenschaften | properties erlaubt definierte Felder |
| 03 | Pflichtfelder | required verhindert unvollständige Nodes |
| 04 | Grenzen | Anzahl und Werte einschränken |
| 05 | Validierung | Schema-Check ausführen |

Bindings beschreiben Hardware als stabile ABI – unabhängig davon, was ein einzelner Treiber unterstützt.

Aus Device-Tree-Knoten entstehen Platform Devices

Device Tree

Root-Knoten

Bus-Knoten

Child mit compatible

of_platform_populate

läuft den Baum ab
und registriert Geräte

Kernelobjekte

platform_device

Name · Ressourcen · of_node

struct device

Parent · Bus · Firmware Node

Population übersetzt beschreibende Knoten in Geräteobjekte des Linux Device Model.

compatible verbindet Platform Device und Platform Driver

```
static const struct of_device_id demo_ids[] = {
    { .compatible = "acme,demo-v1" }, { }
};

MODULE_DEVICE_TABLE(of, demo_ids);
static struct platform_driver demo_drv = {
    .probe = demo_probe,
    .driver.of_match_table = demo_ids,
};

module_platform_driver(demo_drv);
```

01 Match-Tabelle

of_device_id nennt unterstützte Hardware.

02 Registrierung

module_platform_driver registriert Init und Exit.

03 Bindung

Ein Treffer löst probe() für diese Instanz aus.

Die Match-Tabelle ist zugleich Treiberwissen und Grundlage für Modul-Aliase.

probe() übernimmt Ressourcen und initialisiert die Instanz

```
static int demo_probe(struct platform_device *pdev)
{
    void __iomem *base;
    int irq;

    base = devm_platform_ioremap_resource(pdev, 0);
    if (IS_ERR(base))
        return PTR_ERR(base);

    irq = platform_get_irq(pdev, 0);
    if (irq < 0)
        return irq;

    platform_set_drvdata(pdev, base);
    return 0;
}
```

1

Gerät prüfen

Match-Daten und erforderliche Properties prüfen

2

Ressourcen holen

reg, interrupts, clocks und resets übernehmen

3

Zustand speichern

Instanzdaten mit platform_set_drvdata ablegen

4

zuletzt publizieren

Schnittstellen erst nach erfolgreicher Initialisierung

probe() darf mit -EPROBE_DEFER warten, wenn abhängige Provider noch nicht verfügbar sind.

Helfer übersetzen Properties in verwaltete Ressourcen

Helfer	DT-Eigenschaft	Einsatz
devm_platform_ioremap_resource	reg	MMIO abbilden
platform_get_irq	interrupts	IRQ-Nummer erhalten
devm_clk_get	clocks	Taktgeber referenzieren
devm_reset_control_get	resets	Reset-Leitung übernehmen
device_property_read_u32	eigene Property	konfigurierten Wert lesen

Managed APIs vereinfachen Fehlerpfade; betriebliche Reihenfolge und Hardwarezustand bleiben Treiberaufgabe.

Vier Regeln verhindern fragile Platform-Treiber

01 Probe Deferral zulassen

Fehlende Provider mit -EPROBE_DEFER melden.

02 Managed ist nicht vollständig

devm räumt Speicher auf, ersetzt aber kein HW-Shutdown.

03 Device Tree bleibt Hardware

Keine Linux-Policy oder Laufzeitentscheidung kodieren.

04 Remove sicher machen

IRQ, DMA und Hardware vor dem Detach stilllegen.

Robuste Treiber behandeln Abhängigkeiten, Fehlerpfade und wiederholtes Binden als reguläre Lebenszyklen.

Vier Werkzeuge machen Device Tree und Matching sichtbar

```
$ dtc -I fs -O dts /sys/firmware/devicetree/base  
$ find /sys/bus/platform/devices -maxdepth 1 -type l  
$ readlink -f /sys/bus/platform/devices/$DEV/driver  
$ cat /sys/bus/platform/devices/$DEV/modalias  
$ dmesg | grep -i -E 'probe|platform|of:'
```

1

Live Tree

laufenden Baum als DTS ausgeben

2

Platform Bus

Geräte und Treiberinstanzen auflisten

3

Matching

driver-Symlink und modalias prüfen

4

Logs

Probe- und OF-Meldungen untersuchen

Nächste Einheit

Praxis: Platform-Treiber und Device Tree auf einem laufenden System untersuchen